

ARCHITECTURE COURSE

CI/CD

From Commit to Production — Automatically



COMMIT



TEST



PACKAGE



DEPLOY

The World Before: "It Works on My Machine"



Integration Hell

Weeks of isolated work merged at the end — conflicts, broken builds, finger-pointing.



Manual Deployments

A human follows a 14-step Word document at 2 AM. One missed step = production outage.



Late Bug Discovery

Bugs found weeks after the code was written — when context is gone and fixes are expensive.

THE GOAL

**Small,
frequent,
boring releases**

"If it hurts, do it more often."

CI vs. CD vs. CD — Three Levels of Automation



Continuous Integration

Every push is automatically built and tested against the main branch.

Merge small changes daily — find conflicts in minutes, not weeks.



Continuous Delivery

Every passing build is packaged into a deployable artifact.

Production deploy is one button away — a human decides when.



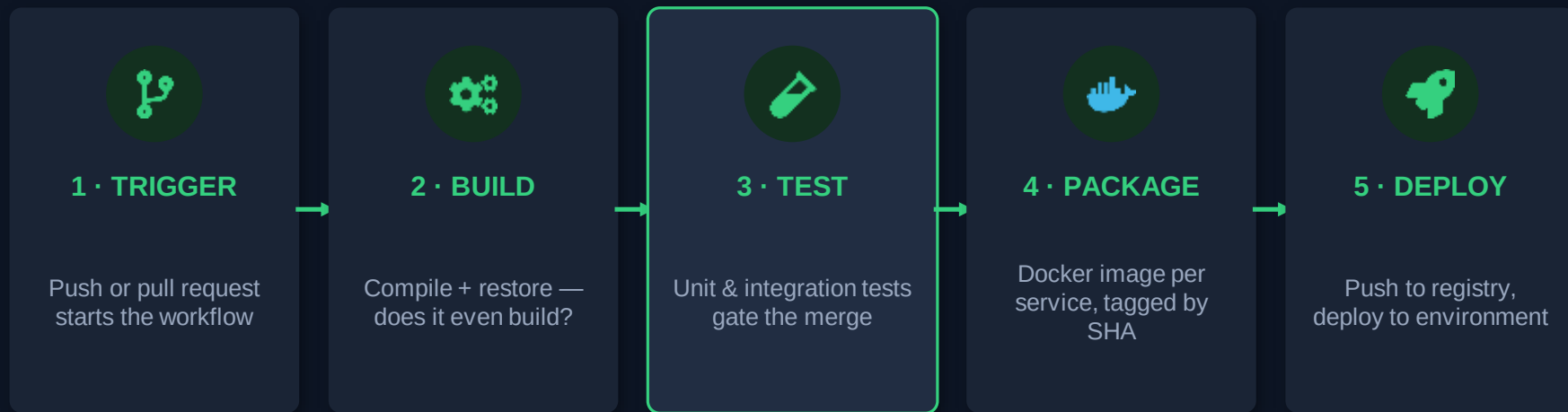
Continuous Deployment

Every passing build goes to production automatically.

No button. The pipeline IS the release process.

Delivery vs. Deployment: the only difference is whether a human presses the final button.

Anatomy of a Pipeline



Fail fast: any red stage stops the pipeline — broken code never moves forward. The TEST stage is your quality gate.

GitHub Actions — The Building Blocks



Workflow

A YAML file in `.github/workflows/` — the whole pipeline definition.



Trigger (on:)

What starts it: `push`, `pull_request`, `schedule`, `manual dispatch`.



Job + Runner

A job is a group of steps that runs on a fresh VM (the runner).



Step

One command or reusable action: `checkout`, `setup-dotnet`, `docker build`...

`build.yml`

```
on: push
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
      - run: dotnet build
      - run: dotnet test
```

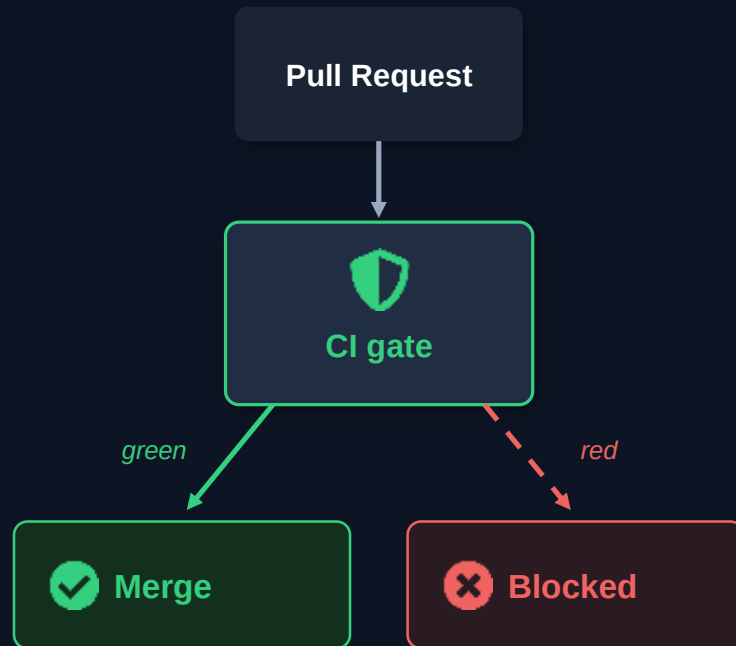
Tests Are the Gatekeeper

A pull request cannot merge until the pipeline is green.

What belongs in the gate:

- Unit tests — fast, run on every push
- Integration tests — against real dependencies in containers
- Static analysis & linting — style and security issues
- Build of every affected service

Tip from our AI lab: generate your xUnit tests with Claude/Copilot — then the pipeline runs them forever.



Docker in the Pipeline



Image per service

Each microservice builds its own image from its own Dockerfile — independently.



Tag by commit SHA

myapp/orders:a1b2c3d — every image traces back to the exact code that produced it. Never deploy :latest.



Push to a registry

Docker Hub, GitHub Container Registry, ACR — the artifact store of your system.



Smoke test

Pipeline runs docker compose up, polls every /health endpoint, tears down. Deploy only what proved it can boot.

Deployment Strategies — Releasing Without Fear



Rolling Update

Replace instances one at a time behind the load balancer.

Zero downtime, slow rollback.



Blue / Green

Two identical environments; flip all traffic at once.

Instant rollback — just flip back.



Canary Release

Send 5% of traffic to the new version; watch metrics; expand.

Real users find bugs before everyone does.

Connects to Resilience: your /health endpoints and monitoring (Ch. Logging) are what make canary and rolling deploys possible.

CI/CD in a Microservices World



Pipeline per service

Each service builds, tests, and deploys independently — that was the whole point of splitting the monolith.



Path filters

In a monorepo, only rebuild what changed: paths: 'src/OrderService/**' — don't rebuild 6 services for a typo in one.



Contract testing

Independent deploys can break consumers. Verify API contracts (e.g., Pact) before a service ships alone.



Shared infra as code

docker-compose / Helm definitions live in Git too — the environment is versioned like the code.

Tooling Landscape & Your Final Project

PICK ANY — JUSTIFY YOUR CHOICE

GitHub Actions

Lives next to your repo — easiest start

GitLab CI

Built-in registry + environments

Azure DevOps

Deep .NET & Azure integration

Jenkins

Self-hosted, infinitely pluggable



FINAL PROJECT — CI/CD BONUS (+5%)

- B.1 Pipeline builds all services on push
- B.2 Unit tests gate the build (reuse your AI-generated xUnit tests)
- B.3 Docker image per service, tagged with the commit SHA
- B.4 Stretch: push to a registry + compose smoke test on /health

Checkpoint: a green badge in your README — and a failing test that visibly blocks the merge.